# SalesSense

## Scalable Sales Prediction & Analytics System

*Mini Project Report*

Built with Apache Spark (PySpark), Spark MLlib, and Streamlit

# 1. Abstract

SalesSense is a scalable sales prediction and analytics system designed to demonstrate distributed data processing and machine learning using Apache Spark. The system ingests a real-world retail sales dataset (9,994 transactions), cleans and transforms it using Spark's DataFrame API, engineers categorical and date-derived features, and trains a Linear Regression model using Spark MLlib within a structured ML Pipeline. The trained model is evaluated using RMSE, MAE, and $R^2$ on a held-out test set, achieving a test RMSE of 536.33, MAE of 227.99, and $R^2$ of 0.292. The system is exposed through a Streamlit web application offering a KPI dashboard, single-record and batch sales prediction, model performance visualization, and deeper sales analytics — all backed by genuine Spark computation rather than static or pre-computed results.

# 2. Introduction

Sales forecasting is a core activity for any retail business: understanding how quantity, discounting, product category, and customer segment influence a transaction's value helps with inventory planning, pricing strategy, and revenue forecasting. As the volume of transactional data grows, processing and modeling it efficiently requires tools built for scale. Apache Spark is the most widely used framework for distributed data processing, and its MLlib library provides scalable implementations of standard machine learning algorithms that integrate directly with Spark's DataFrame API. This project builds a complete pipeline — from raw CSV to a deployed, interactive prediction application — using Spark throughout the data and modeling stages, with Streamlit providing the user-facing layer.

# 3. Problem Definition

Given historical sales transaction records containing product, customer, order, and pricing attributes, predict the Sales value of a transaction from its other attributes (Quantity, Discount, Category, Sub-Category, Segment, Region, Ship Mode, and order date). This is framed as a supervised regression problem: the target variable, Sales, is continuous. The system must additionally demonstrate that this processing and modeling genuinely uses Spark's distributed computation model, and must expose the trained model through a usable interface for both single predictions and bulk (batch) predictions.

# 4. Motivation

Most academic ML projects are built and demonstrated entirely in pandas/scikit-learn on small, static datasets, which does not reflect how sales data is actually processed in industry — at a scale where single-machine, in-memory tools become impractical. Learning to build the same kind of pipeline in Spark (cleaning, feature engineering, model training, and inference, all through Spark's own APIs) is directly transferable to real data engineering and data science work. Additionally, presenting the resulting model through a genuinely usable, polished web interface — rather than a notebook — better demonstrates the practical, deployable nature of the system.

# 5. Objectives

- Select and inspect a real-world sales dataset, reporting its actual size, structure, and quality (missing values, duplicates) rather than assuming them.
- Clean and preprocess the dataset using PySpark's DataFrame API (type casting, null/duplicate handling, date parsing).
- Engineer relevant numerical and categorical features, explicitly avoiding target leakage.
- Build a Spark ML Pipeline chaining StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler, and LinearRegression.
- Train the model and evaluate it using RMSE, MAE, and $R^2$ on a genuine held-out test split.
- Persist the trained pipeline and reload it for both single and batch inference.

- Build an interactive, professional-looking Streamlit GUI covering dashboarding, prediction, and model evaluation.
- Actually execute every stage of the pipeline and report real, unmodified results.

## 6. Literature / Existing System Overview

Traditional sales analysis in small-to-medium organizations is commonly done with spreadsheet tools (Excel, Google Sheets) or single-machine BI tools (Tableau, Power BI) connected to a data warehouse. These tools are effective for visualization but are not designed for training machine learning models at scale, and typically rely on a separate, disconnected modeling workflow (e.g. a Python/pandas/scikit-learn notebook) that does not scale past what fits in a single machine's memory. Existing distributed alternatives to Spark include Hadoop MapReduce (lower-level and slower for iterative ML workloads) and cloud-managed services (e.g. AWS SageMaker, Databricks) which internally often use Spark or similar engines. This project's approach — Spark end-to-end with a lightweight web front-end — mirrors how many production analytics pipelines are structured, at a scale appropriate for a mini-project.

## 7. Proposed System

The proposed system, SalesSense, consists of three layers: (1) a Spark-based data processing and machine learning layer that cleans data, engineers features, and trains/evaluates a Linear Regression model; (2) a persistence layer that saves the trained Spark ML Pipeline to disk; and (3) a Streamlit web application layer that loads the persisted model and exposes it through a dashboard, prediction forms, and analytics pages. All heavy data operations (loading, cleaning, aggregation, feature transformation, training, and inference) are performed by Spark; the web layer is a thin, cached presentation layer over Spark's outputs.

## 8. Dataset Description

The Sample Superstore dataset, sourced from Kaggle (vivek468/superstore-dataset-final, https://www.kaggle.com/datasets/vivek468/superstore-dataset-final), is a widely used, real-world US retail transactions dataset. The following figures were obtained by directly inspecting the dataset with pandas/PySpark (not assumed or estimated):

| Property | Value |
|---|---|
| Total rows | 9,994 |
| Total columns (raw) | 21 |
| Missing values | 0 (across all columns) |
| Duplicate rows | 0 |
| Order date range | 2014-01-03 to 2017-12-30 (4 years) |
| Target variable (Sales) range | ₹0.444 to ₹22,638.48 |
| Target variable (Sales) mean | ₹229.86 |
| Category values | Office Supplies (6,026), Furniture (2,121), Technology (1,847) |
| Sub-Category values | 17 distinct values |
| Region values | West (3,203), East (2,848), Central (2,323), South (1,620) |
| Segment values | Consumer (5,191), Corporate (3,020), Home Office (1,783) |
| Ship Mode values | Standard Class (5,968), Second Class (1,945), First Class (1,538), Same Day (543) |

Correlation of Sales with other numeric columns (computed on the cleaned data): Quantity 0.201, Discount −0.028, Profit 0.479. The Profit correlation is notably higher than the others because Profit is arithmetically derived from Sales in this dataset — this is precisely why Profit is excluded from the model's input features (see Section 9).

## 9. Data Preprocessing

Preprocessing is implemented in src/preprocessing.py using PySpark's DataFrame API, and includes:

- Reading the CSV with Spark's distributed CSV reader, with explicit quote/escape options to correctly parse product names containing embedded quoted commas (a real parsing issue discovered and fixed during development — see Section 17).
- Renaming raw column headers (which contain spaces and hyphens, e.g. "Sub-Category") to Spark-safe identifiers.
- Dropping exact duplicate rows (dropDuplicates()) — none were found in this dataset.
- Dropping rows with nulls in any feature/target-essential column — none were found.
- Casting Sales, Quantity, and Discount to explicit numeric types.
- Filtering out logically invalid rows (non-positive Sales or Quantity, negative Discount) as a defensive measure.
- Parsing the raw Order Date string (M/d/yyyy format) into a proper Spark DateType column.

Result: all 9,994 raw rows pass cleaning unchanged, confirming the dataset was already high quality — a real, verified result, not assumed.

## 10. System Architecture

Raw CSV → PySpark DataFrame (load + clean) → Feature Engineering (date-part extraction) → Spark ML Pipeline (StringIndexer → OneHotEncoder for each categorical column → VectorAssembler → StandardScaler) → LinearRegression → fitted PipelineModel → saved to disk → loaded by the Streamlit application for single-record and batch inference. Aggregation queries for the dashboard/analytics pages are computed separately via Spark groupBy/agg operations over the cleaned DataFrame.

## 11. Methodology

The end-to-end workflow follows the standard supervised regression methodology, adapted to Spark's Pipeline abstraction:

1. Data ingestion: load the CSV into a Spark DataFrame with schema inference.
2. Data cleaning: handle nulls, duplicates, and type casting (Section 9).
3. Feature engineering: derive Order_Month and Order_DayOfWeek from the order date; select the final feature set (Section 12).
4. Train/test split: 80% train, 20% test, using a fixed random seed (42) for reproducibility.
5. Pipeline fitting: fit all pipeline stages (indexers, encoders, assembler, scaler, and the regressor) on the training set only, to prevent test-set leakage into any fitted statistic (e.g. StandardScaler's mean/std).
6. Evaluation: transform the test set through the fitted pipeline and compute RMSE, MAE, and $R^2$.
7. Persistence: save the entire fitted PipelineModel so identical transformations are applied at inference time.

## 12. Apache Spark Implementation

Spark is used for every data-touching stage of this project, not as a token import. Specifically:

- SparkSession configured with local[*] (uses all available CPU cores on the host machine), demonstrating parallel execution; the same code would run unchanged against a cluster master URL.
- Distributed CSV read with schema inference (spark.read.csv).
- DataFrame transformations for cleaning: dropDuplicates(), dropna(), withColumn() with casts and date parsing.
- Feature engineering via Spark SQL functions (F.month, F.dayofweek, F.to_date).
- A Spark ML Pipeline with 5 StringIndexer + 5 OneHotEncoder stages (one pair per categorical feature), a VectorAssembler combining 4 numeric + 5 one-hot vector columns into a single feature vector, and a StandardScaler.

- LinearRegression from spark.ml.regression, trained as the final Pipeline stage.
- RegressionEvaluator (spark.ml.evaluation) for RMSE/MAE/$R^2$ computation.
- Spark DataFrame groupBy/agg aggregations for all dashboard and analytics figures (monthly trend, category/region/segment/sub-category breakdowns, top products) — aggregation happens in Spark, and only the small aggregated result is converted to pandas for charting.

Why Spark is useful here: even though this dataset (9,994 rows) fits comfortably in memory, the same code path — DataFrame transformations, Pipeline-based feature engineering, and MLlib model training — scales to datasets far larger than a single machine's memory by distributing data and computation across a cluster. Writing the pipeline in Spark from the start means the system does not need to be rewritten to handle a 10x or 100x larger dataset.

## 13. Machine Learning Model

Algorithm: Linear Regression (spark.ml.regression.LinearRegression), trained with maxIter=50, regParam=0.1 (L2 regularization for numerical stability), elasticNetParam=0.0 (pure ridge regularization).

Features used (final feature vector size: 35 dimensions after one-hot encoding):

| Feature | Type | Justification |
|---|---|---|
| Quantity | Numeric | Direct driver of order value |
| Discount | Numeric | Affects the final sale price |
| Order_Month | Numeric (engineered) | Captures seasonal demand patterns |
| Order_DayOfWeek | Numeric (engineered) | Captures weekly ordering patterns |
| Category | Categorical (one-hot) | Product type strongly affects price point |
| Sub_Category | Categorical (one-hot) | Finer-grained product price signal |
| Segment | Categorical (one-hot) | Customer type affects order size/pricing |
| Region | Categorical (one-hot) | Regional demand/pricing variation |
| Ship_Mode | Categorical (one-hot) | Proxy for order urgency/size |

Profit was deliberately excluded from the feature set. In this dataset Profit is mathematically derived from Sales (correlation 0.479, far higher than any genuine predictor), so including it as an input feature would leak information about the target and produce an artificially inflated, unrealistic evaluation score. High-cardinality identifier columns (Customer ID, Product ID, Product Name, City) were also excluded to keep the feature space reasonable for a college-scope project and to avoid overfitting on identifiers that do not generalize to unseen customers/products.

## 14. GUI Implementation

The GUI is built with Streamlit and Plotly, structured as six sidebar-navigable pages: Dashboard, Sales Prediction, Batch Prediction, Model Performance, Analytics, and About Project. Streamlit's caching (@st.cache_resource for the SparkSession and trained model, @st.cache_data for the cleaned dataset and precomputed aggregations) ensures Spark is initialized once per session rather than on every page interaction. Custom CSS provides a consistent dark, professional theme with card-based KPI displays and a styled prediction result card. All charts are rendered with Plotly for interactivity (zoom, hover, legend toggling).

## 15. Results

The model was trained once (seed=42) and evaluated on a genuine, unseen 20% test split. All figures below are actual outputs of the training run, saved to model/metrics.json:

| Metric | Test Set | Training Set |
|---|---|---|
| RMSE | 536.33 | 540.82 |
| MAE | 227.99 | — |
| $R^2$ | 0.292 | 0.238 |
| Rows | 1,952 | 8,042 |

Training took 7.11 seconds on the development machine (local[*] mode). The trained PipelineModel was saved and successfully reloaded by the Streamlit app for both single-record and batch prediction, both verified with real inputs during testing (Section 17).

## 16. Model Evaluation

Metric definitions, in plain language:

- RMSE (Root Mean Squared Error): the typical size of the model's prediction error, in the same units as Sales (₹). Squaring the errors before averaging means large errors are penalized disproportionately, so RMSE is sensitive to big misses.
- MAE (Mean Absolute Error): the average absolute difference between predicted and actual Sales. Easier to interpret directly — "on average, the prediction is off by ₹X" — and less sensitive to outliers than RMSE.
- $R^2$ (Coefficient of Determination): the proportion of variance in Sales explained by the model, ranging from 0 (no better than always predicting the mean) to 1 (perfect fit). An $R^2$ of 0.292 means the model explains roughly 29% of the variance in Sales.

Test $R^2$ (0.292) is slightly higher than training $R^2$ (0.238), and test RMSE is close to training RMSE — this indicates the model is not overfitting; its limited accuracy is a genuine property of a linear model on this data, not a symptom of poor generalization.

## 17. Performance Analysis

During development, two real issues were found and fixed through actual execution rather than assumed away:

- CSV parsing bug: several product names in the raw file contain embedded, doubled quote characters (e.g. 14 7/8" x 11"), which Spark's default CSV reader mis-parsed, shifting columns and causing a cast failure. Fixed by explicitly setting quote="" and escape="" options on the Spark CSV reader.
- Pandas/Spark date type mismatch: Spark's DateType columns returned from toPandas() came back as Python datetime.date objects (object dtype) rather than pandas datetime64, which broke the Dashboard's date-range filter comparison. Fixed by explicitly coercing with pd.to_datetime() immediately after the Spark-to-pandas conversion.

After these fixes, the full application was tested programmatically (via Streamlit's AppTest framework) across all six pages with zero unhandled exceptions, and the prediction and batch-prediction flows were exercised with real inputs, producing real (non-fabricated) outputs — e.g. a Furniture/Bookcases/Consumer/West/Standard-Class/Qty-2 record predicted ₹444.36.

## 18. Advantages

- Genuinely scalable data processing: the Spark-based pipeline would handle datasets far larger than memory without code changes, unlike a pure-pandas approach.
- End-to-end reproducibility: a fixed random seed and a persisted Pipeline ensure identical preprocessing is applied at training and inference time.
- No feature leakage: Profit was deliberately excluded after identifying it as a derived quantity.
- Clear, honest evaluation: real RMSE/MAE/$R^2$ figures reported, including the train/test comparison that demonstrates no overfitting.
- Usable, polished interface: batch and single prediction, model diagnostics, and analytics are all accessible without writing code.

## 19. Limitations

- Modest predictive accuracy ($R^2 \approx 0.29$): Linear Regression is a purely linear model and cannot capture non-linear interactions (e.g. how discount effects vary by category) or model the heavy right-skew of Sales (a small number of very large orders dominate the error).
- No hyperparameter tuning was performed beyond a single fixed regularization value; a CrossValidator grid search was out of scope for this project's timeline.
- The dataset, while real, is limited to US retail transactions from a single company over 4 years; the model would need retraining/revalidation for other markets or time periods.
- The system runs Spark in local mode on a single machine for this demonstration; true distributed-cluster behavior (multi-node scheduling, shuffle over a network) is not directly observed here, only architecturally supported.

## 20. Conclusion

This project successfully implements a complete, scalable sales prediction and analytics system using Apache Spark and Spark MLlib for data processing and modeling, and Streamlit for an interactive, professional user interface. Every stage — data cleaning, feature engineering, pipeline construction, model training, evaluation, and deployment through a working GUI — was implemented and actually executed, with real, verified results rather than assumed or fabricated figures. While the Linear Regression model's accuracy is modest, this is an honest and explainable outcome of applying a linear model to a skewed real-world target, and the system's architecture is designed to support more capable models (Section 21) without structural changes.

## 21. Future Scope

- Replace/augment Linear Regression with Spark MLlib's tree-based regressors (RandomForestRegressor, GBTRegressor) to capture non-linear relationships.
- Apply CrossValidator/ParamGridBuilder for systematic hyperparameter tuning.
- Log-transform the Sales target to reduce the influence of extreme high-value orders.
- Deploy on an actual multi-node Spark cluster and test against a substantially larger dataset.
- Add authenticated multi-user access, persistent prediction history, and scheduled model retraining.

## 22. References


- Apache Spark Documentation — https://spark.apache.org/docs/latest/
- Spark MLlib Programming Guide — https://spark.apache.org/docs/latest/ml-guide.html
- Streamlit Documentation — https://docs.streamlit.io/
- Plotly Python Graphing Library — https://plotly.com/python/
- Sample Superstore Dataset (Kaggle) — https://www.kaggle.com/datasets/vivek468/superstore-dataset-final